Code Jam 2012 - Round 1B

Analysis: Equal Sums

The Small Input

If you want to forget about the large input and go straight for the small, then Equal Sums might look like a classic dynamic programming problem. Here is a sketch of one possible solution in Python:

```
def GetSetWithSum(x, target):
  if target == 0: return []
  return GetSetWithSum(x, target - x[target]) + [x[target]]

def FindEqualSumSubsets(S):
  x = [0] + [None] * (50000 * 20)
  for s in S:
    for base_sum in xrange(50000 * 20, -1, -1):
      if x[base_sum] is not None:
        if x[base_sum + s] is None:
          x[base_sum + s] = s
        else:
          subset1 = GetSetWithSum(x, base_sum + s)
          subset2 = GetSetWithSum(x, base_sum) + [s]
          return subset1, subset2
  return None
```

The idea is that, in x, we store a way of obtaining every possible subset sum. If we reach a sum in two different ways, then we can construct two subsets with the same sum.

For the small input, this approach should work fine. However, x would have to have size 500 * 10^{12} for the large input. That is too big to work with, and you will need a different approach there.

The Large Input

The first step towards solving the large input is realizing that the statement is very misleading! Let's suppose you have just 50 integers less than 10^{12}. There are 2^{50} ways of choosing a subset of them, and the sum of the integers in any such subset is at most $50 * 10^{12} < 2^{50}$. By the Pigeonhole Principle, this means that there are two different subsets with the same sum.

So what does that mean for this problem? Well, even if you had just 50 integers to choose from instead of 500, the answer would never be "Impossible". You need to find some pair of subsets that have the same sum, and there is a lot of slack to work with. The trick is figuring out how to take advantage of that slack.

The Birthday "Paradox" Method

The simplest solution is based on a classic mathematical puzzle: the birthday paradox.

The birthday paradox says that if you have just 23 people in a room, then some two of them probably have the same birthday. This can be surprising because there are 365 possible days,

and 23 is much smaller than 365. But it is true! One good way to look at is this: there are 23 choose 2 = 253 pairs of people, and each pair of people has a 1/365 chance of having the same birthday. In particular, the *expected* (aka average) number of pairs of people with the same birthday is 253 / 365 = 0.693... Once you write it that way, it is not too surprising that the probability of having at least *one* pair matching is about 0.5.

It turns out this exact reasoning applies to the Equals Sums problem just as well. Here is a simple algorithm:

- Choose 6 random integers from **S**, add them up, and store the result in a hash set. (Why 6? We'll come back to that later...)
- Repeat until two sets of 6 integers have the same sum, then stop.

After choosing t sets, there will be t choose 2 pairs, and each set will have sum at most $6 * 10^{12}$. Therefore, the expected number of collisions would be approximately $t^2 / (12 * 10^{12})$. When t is around 10^6, this expectation will be near 1, and the reasoning from the birthday paradox says that we'll likely have our collision.

That's it! Since we can quickly generate 10^6 small subsets and put their sums into a hash-set, this simple algorithm will absolutely solve the problem. You may be worried about the randomness, but for this problem at least, you should not be. This algorithm is extremely reliable. By the way, this method will work on the small input as well, so you don't need to do the dynamic programming solution if you do not want to.

Note: There are many similar approaches here that can work. For example, in our internal tests, one person solved the problem by only focusing on the first 50 integers, and trying random subsets from there. The proof below works for this algorithm, as well as many others.

A Rigorous Proof

If you have some mathematical background, we can actually prove rigorously that the randomness is nothing to worry about. It all comes down to the following version of the birthday theorem:

Lemma: Let X be an arbitrary collection of N integers with values in the range [1, R]. Suppose you choose t + 1 of these integers independently at random. If N ≥ 2R, then the probability that these randomly chosen integers are all different is less than $e^{-t^2 / 4R}$.

Proof: Let x_i denote the number of integers in X with value equal to i. The number of ways of choosing t + 1 distinct integers from X is precisely

$$\text{sum}_{(1 \le i_1 < i_2 < \ldots < i_{t+1} \le R)} \left[x_{i_1} * x_{i_2} * \ldots * x_{i_{t+1}} \right].$$

For example, if t=1 and R=3, the sum would be $x_1 * x_2 + x_1 * x_3 + x_2 * x_3$. Each term here represents the number of ways of choosing integers with a specific set of values. Unfortunately, this sum is pretty hard to work with directly, but Maclaurin's inequality states that it is at most the following:

$$(R \text{ choose } t+1) * \left[(x_1 + x_2 + \ldots + x_R) / R \right]^{t+1}$$
$$= (R \text{ choose } t+1) * (N/R)^{t+1}.$$

On the other hand, the number of ways of choosing *any* t + 1 integers out of X is equal to $N \text{ choose } t+1$. Therefore, the probability p that we are looking for is at most:

$$[(R \text{ choose } t+1) / (N \text{ choose } t+1)] * (N/R)^{t+1}$$
$$= R/N * (R-1)/(N-1) * (R-2)/(N-2) * \ldots * (R-t)/(N-t) * (N/R)^{t+1}.$$

Now, since N ≥ 2R, we know the following is true for all a ≤ t:

$$(R-a) / (N-a)$$
$$< (R - a/2)^2 / [R * (N-a)] \quad \text{(this is because } (R - a/2)^2 \geq R(R-a))$$
$$\leq (R - a/2)^2 / [N * (R-a/2)]$$
$$= (R - a/2) / N.$$

Therefore, p is less than:

$$R * (R - 1/2) * (R - 2/2) * \ldots * (R - t/2) / R^{t+1}.$$

It is now easy to check that $(R-a/2) * (R-t/2+a/2) \leq (R-t/4)^2$, from which it follows that p is also less than:

$$(R - t/4)^{t+1} / R^{t+1}$$
$$= (1 - t/4R)^{t+1}$$
$$< (1 - t/4R)^{t}.$$

And finally, we use the very useful fact that $1 - x \leq e^{-x}$ for all x. This gives us $p < e^{-t^2 / 4R}$ as required.

In our case, X represents the sums of all 6-integer subsets. We have N = 500 choose 6 and R = $6 * 10^{12}$. You can check that N ≥ 2R, so we can apply the lemma to estimate the probability that the algorithm will still be going after t+1 steps:

- If t = 10^6, the still-going probability is at most 0.972604477.
- If t = $5*10^6$, the still-going probability is at most 0.499351789.
- If t = 10^7, the still-going probability is at most 0.062176524.
- If t = $2*10^7$, the still-going probability is at most 0.000014945.
- If t = $3*10^7$, the still-going probability is at most 0.00000000001.

In other words, we have a good chance of being done after 5,000,000 steps, and we will *certainly* be done after 30,000,000 steps. Note this is a mathematical fact, regardless of what the original 500 integers were.

Comment: What happens if you look at 5-element subsets instead of 6-element subsets? The mathematical proof fails completely because N < R. In practice though, it worked fine on all test data we could come up with.

A Deterministic Approach

The randomized method discussed above is certainly the simplest way of solving this problem. However, it is not the only way. Here is another way, this time with no randomness:

- Take 7,000,000 3-integer subsets of **S**, sort them by their sum, and let the difference between the two closest sums be d_1. Let X_1 and Y_1 be the corresponding subsets.
- Remove X_1 and Y_1 from **S**, and repeat 25 times to get X_i, Y_i and d_i for i from 1 to 25.

- Let $Z = \{d_1, d_2, ..., d_{25}\}$. Calculate all 2^{25} subset sums of Z.
- Two of these subset sums are guaranteed to be equal. Find them and trace back through X_i and Y_i to find two corresponding subsets of **S** with equal sum.

There are two things to show here, in order to justify that this algorithm works:

- ***S** will always have at least 7,000,000 3-integer subsets.* This is because, even after removing X_i and Y_i, **S** will have at least 350 integers, and 350 choose 3 > 7,000,000.
- *Z will always have two subsets with the same sum.* First notice that the subsets in the first step have sum at most $3 * 10^{12}$, and so two of the sums must differ by at most $3 * 10^{12} / 7{,}000{,}000 < 500{,}000$. Therefore, each d_i is at most 500,000. Now, Z has 2^{25} subsets, and each has sum at most $25 * 500{,}000 < 2^{25}$, and so it follows from the Pigeonhole Principle that two subsets have the same sum.

Other similar methods are possible too. This is a pretty open-ended problem!

Code Jam 2012 - Round 1B

Analysis: Safety in Numbers

Introduction

Like some earlier problems in Code Jam 2012, this one was inspired by the author watching significant amounts of Dancing With the Stars. Unlike previous problems, this one needed no modification to be used in Code Jam (although we don't know what they do in case of a tie): this is exactly how their scoring system works.

Sample Cases

Sometimes it's useful when solving Code Jam problems to start by working to understand the provided sample cases. Let's start there.

Case #1

In the first case, if the first dancer (err, contestant) gets 1/3 of the audience votes, then his score is 30. There's only one way for the rest of the votes to be distributed, which is for 2/3 of the votes to go to the second contestant. That contestant's score would then be 30, which is a tie, and would lead to contestant #1 being safe. Any lower score would result in that contestant being eliminated. We can do similar math on the second contestant.

Case #4

Now let's think about a case where there are more than two contestants. In the fourth sample case, there are three contestants, and their judges' scores are 24, 30 and 21. The sample output claims that the first contestant is safe if she gets 34.666667% of the vote. Is that really true?

That proportion of the votes would give contestant #1 a total of 50 points. For her to be eliminated, all other contestants would need more than 50 points. We can do a little math and find out that for the second contestant to get more than 50 points, he would need more than 26.66666...% of the vote. For the third contestant to get more than 50 points, she would need more than 38.66666...% of the vote. Adding those numbers up gives us 100%.

That means the other contestants would need more votes than the 65.333333% that remain in order to all beat the first contestant, so the first contestant can't be eliminated. If we lowered the first contestant's percentage at all, then the other contestants would all be able to beat her, and she'd end up being eliminated; so 34.666667% is correct.

N Binary Searches

Considering the sample cases has led us very naturally to an algorithm. When we're trying to find the minimum score that will make a particular contestant safe, first we'll choose that contestant's audience vote percentage, and then we'll allocate the rest of the audience votes to other contestants in the worst possible way.

This lends itself very naturally to **binary search**. First we pick a percentage. Is the contestant safe at that percentage? If so, the minimum safe percentage is lower. If not, the minimum safe percentage is higher. Repeat that **N** times, and you have the answer for each contestant.

Those numbers add up to 1!

It's a bit surprising that, when you independently calculate the minimum safe percentage for each contestant, you apparently end up with percentages that add up to 1. A little thought shows us why.

If each contestant has a minimum "safe" percentage, we can say that contestant has a minimum "safe" score (50 for contestant #1 in case #4). Thinking about the last example, we might say the minimum safe score is the value `x` such that we use up 100% of the audience votes if all contestants have a score equal to `x`. The percentages that contestants need to reach that score will naturally add up to 100%.

This is almost true, but there is one exception. Consider the following case: `4 30 0 0 0`. The safe score clearly won't be more than 30, since not all contestants can get to 30, so we have to deal specially with contestants whose score is greater than the safe score. With that in mind, the safe score is the value `x` such that we use up 100% of the audience votes if all contestants have either a score equal to `x`, or 0% of the audience votes and a score greater than `x`.

A Different Binary Search

Now we know how to write a single binary search to find the "safe" score. The safe score is the score for which all contestants with a higher number of judge points get 0% of the audience votes; the other contestants get a percentage that gives them that total score; and the total audience vote percentages add up to 100%.

So to find the safe score, we can try a candidate score, and see if the total audience vote percentage adds up to more than 100%. If so, we need a lower score. If not, we need a higher score. As one of our preparers put it, "Binary search for the number such that it's impossible for all contestants to have that many points, and you're done!"

A Linear Solution

After this editorial was first published, a number of contestants wrote in to tell us that they didn't use binary search at all. One way to avoid it is to use a single loop, plus some math, to compute the "safe" score we talked about before. First, create a sorted list of judges' points. Then, for `i` from 0 to **N**-1, repeatedly determine the following number: if we allocated all the audience votes to the first `i` people to give them the same score, what would that score be? If it's less than the score for contestant `i+1`, or `i+1=`**`N`**, then that's the "safe" score.

Precision

The output of this problem had to be a floating-point number, and any number would be fine as long as it was within 1e-5 of the correct number, absolute or relative. What does that mean?

- **Absolute:** If the correct answer is `y` and you output `x`, you will be judged correct if $|y-x| < 10^{-5}$.
- **Relative:** If the correct answer is `y` and you output `x`, you will be judged correct if $|1-\min(y/x,x/y)| < 10^{-5}$.

We confused some of our contestants by outputting an inconsistent number of decimal places in our sample output. The reason we did that was to illustrate that the number of decimal places you output isn't important, as long as the answer was right; unfortunately, although a number of contestants got an impromptu education on this rule, many more were confused. We'll think about careful ways of presenting problems with floating-point output in the future.

Solving the Small

There's actually an algorithmic approach for this problem that isn't good enough to solve the Large, but does work for the Small, if you aren't up on your binary search. Because each answer you come up with only has to be within 1e-5 of the correct answer, and we know the right answer is between 0 and 100, there are only a couple million numbers you have to check for each user's minimum score that avoids elimination.

Check 0, 0.00001, 0.00002, 0.00003, ... 9.99999, 10. That's 1 million numbers. Then, because the right answer can be within 1e-5 multiplicatively, and the answers we're checking now are more than 10, you can check 10.0001, 10.0002, ..., 99.9999, 100. That's another 900,000 numbers. So by checking 1.9 million numbers, we avoid having to implement binary search; though to be honest, it's probably easier to implement binary search than do that -- and binary search only has to check $\log_2(10^5)=17$ numbers!

Also, note that this method will only work to replace the first binary search we talked about here, not the second one. It isn't enough that the safe score is within 1e-5 of the correct value: the numbers we *output* have to be that close.

Why did people get it wrong?

5608 people downloaded the Small input for this problem, but only 2687 of them got it right. That's an unusually low success rate. So what did those people do wrong?

Some of them outputted a negative percentage for users with a score above the "safe score". Presumably that would have moved the safe score for them as well. Users who did that, and were told that they'd gotten the wrong answer, could have checked their output to see that they'd printed a negative number. In Code Jam you have access to the input and output file you're being tested on; use it!

Others made assumptions that ended up not being true about certain derived values being integers. One contestant who did that got our sample cases right, but outputted `0.0 33.0 33.0 33.0` for the case `4 10 0 0 0`. Still others had problems with integer division: in many programming languages an integer divided by an integer will always return an integer. If you want to avoid that, you have to "cast" one of the integers to a floating-point number -- or just add 0.0 to it, which amounts to the same thing.

Some users might have made a mistake early, then fixed it, but submitted the output for the wrong input file. Whenever you retry a submission, it's important to run your code on the input you just downloaded, or you'll get the wrong answer! We're working on ways to make it easier for users to catch this problem.

Code Jam 2012 - Round 1B

Analysis: Tide Goes In, Tide Goes Out

There is a lot going on in this problem, and it is difficult to keep track of everything. Only seven hundred contestants managed to get it right, so you know it can't have been easy! Let's go over it.

The first thing you should notice is that this is a shortest-path problem. We want to get from point A (the start) to point B (the exit) as fast as possible. There are a number of classical shortest-path algorithms available; we will try to adapt Dijkstra's algorithm here.

Dijkstra's algorithm works on a weighted graph, so we need to identify a set of vertices and edges between them to use it. This is pretty easy if you have done graph theory before - we take the squares to be the vertices, and we put an edge between every two adjacent cells. We encounter problems when trying to assign weights to the edges: the cost (in seconds) of travelling from one square to another is not fixed - it can be one or ten seconds (and we haven't touched on the issue of moving around before the tide starts going down yet).

A fact that might be somewhat surprising is that this is not a problem for Dijkstra's algorithm. Let us briefly recall how it works - it maintains a tentative cost for each vertex, and at each step it chooses the vertex with the smallest cost, fixes that to be the real cost of visiting that vertex, and updates the neighbors accordingly. Of course in our case the cost of reaching a vertex is simply the time it takes.

This means that we need to consider the time of going from some square **A** to an adjacent square **B** only once, when we are recalculating the time of reaching **B** due to having fixed the time for **A**. But since we fixed the time of reaching **A**, we can easily calculate the water level at this moment - and so we will know the cost of travelling this edge. (Note that the earlier we can start moving from **A**, the faster the move will be. Therefore, we really should be moving as soon as possible instead of waiting for a better moment.)

There are also other constraints that we have. Let's name two of them - the water level needs to be at least 50 centimeters below the ceiling of the square we want to enter, and the "gap condition" needs to be satisfied. For the latter, we can just check - this does not depend on the time we want to travel - and remove the edge from the graph if the condition is not satisfied (we can either do this up-front, or lazily at the moment we try to use this edge). For the former, we need to look at what Dijkstra's algorithm needs again. The cost of travelling the edge **AB** is used to determine the shortest path to **B** that goes through **A** and then directly through the edge **AB** to **B**. Well, if we really want to take this path, and the water level is too high to enter **B** when we arrive at **A**, we have only one choice - we have to wait until the water level drops to $\mathbf{C_B}$-50 before moving. Remember that this may cause us to have to drag the kayak!

All this means that we can calculate the cost of each edge at the moment in which we need it, and we would have a complete solution for the problem if not for the extra possibility of moving around before the tide starts going down. One might be tempted to solve this issue by a preprocessing phase, where we use a breadth-first search to find all the squares that are reachable from the start in time zero.

A simpler solution to code, however, is to insert this phase into the Dijkstra's algorithm that works so well for us so far. All this extra movement means is that if we want to traverse the edge **AB**, we are at **A** in time zero, and **B** is accessible from **A** in time zero, then the cost of making this move is zero - as we can make it before the tide starts going down. This means that we insert one extra condition in our function that calculates the cost, and we are done!

Notice that the outcome here is just a standard implementation of Dijkstra's algorithm and nothing more, with all the problem-specific logic inserted into the function that calculates the weight of a given edge at the time it is needed. To convince oneself that this works, however, an understanding of the inner workings of the algorithm is needed.